

LangSmith Cheatsheet

Kurulum, ilk trace, temel terimler (Run/Trace/Thread/Trajectory), Observability vs Evaluation.

pip install langsmith · @traceable · Client · emineksknc.github.io/langsmith-turkce

1. Kurulum & İlk Trace

Ortam değişkenleri

```
LANGSMITH_TRACING=true  
LANGSMITH_API_KEY=ls_...  
LANGSMITH_PROJECT=ilk-projem
```

Eski isimler (LANGCHAIN_TRACING_V2, LANGCHAIN_API_KEY) hâlâ çalışır.

LangChain/LangGraph: sıfır kod

```
from langchain_openai import ChatOpenAI  
  
llm = ChatOpenAI(model="gpt-4o-mini")  
llm.invoke("Merhaba")  
# otomatik trace olur
```

@traceable — LangChain'siz kod için

```
from langsmith import traceable  
  
@traceable  
def cevap_uret(soru: str) -> str:  
    return f"cevap: {soru}"
```

wrap_openai — OpenAI SDK için

```
from openai import OpenAI  
from langsmith.wrappers import wrap_openai  
  
client = wrap_openai(OpenAI())  
client.chat.completions.create(...)
```

İç içe @traceable fonksiyonları otomatik parent-child gösterir.

2. Temel Terimler

Run · Trace · Thread · Trajectory

- Run** — tek bir işlemin kaydı (LLM çağırısı, araç)
- Trace** — kök run'dan başlayan tüm run ağacı
- Thread** — birden fazla trace'i gruplayan konuşma kimliği (LangGraph'taki thread_id ile aynı)
- Trajectory** — ajanın izlediği adım dizisi (hangi araç, hangi sırayla)

Hiyerarşi

```
Thread (konuşma)  
├── Trace 1, Trace 2, ...  
│   ├── Run'lar (LLM/tool çağrıları)  
│   └── Trajectory (araç sırası)
```

3. Observability vs Evaluation

	OBSERVABILITY	EVALUATION
Soru	Üretimde ne oluyor?	Değişiklik kaliteyi bozdu mu?
Veri	Gerçek trafik	Sabit dataset
Ne zaman	Sürekli	Deploy öncesi / örneklenmiş

Pratik döngü: Observability'den düşük skorlu örnekleri bulup Evaluation dataset'ine ekle — dataset zamanla gerçek dünyaya benzesin.

4. Hızlı Referans

KAVRAM	NE İŞE YARAR
@traceable	Fonksiyonu izlenebilir yapar
wrap_openai	OpenAI istemcisini otomatik izler
Client	LangSmith API'sine erişim
LANGSMITH_PROJECT	Trace'lerin gruplanacağı proje